La programmation orientée objet

Capacités attendues

- ✓ Écrire la définition d'une classe.
- ✓ Accéder aux attributs et méthodes d'une classe.
- ✓ Utiliser des API (*Application Programming Interface*) ou des bibliothèques.
- ✓ Exploiter leur documentation.
- ✓ Créer des modules simples et les documenter.

Toute variable informatique possède un type (en python : `bool`, `int`, `float`, `str`, `list`, `dict`, *etc.*) qui indique à l'interpréteur comment la représenter en mémoire et quelles sont les opérations ou méthodes permises.
Par exemple :

- les entiers, les flottants ou les caractères ne sont pas codés de la même manière en mémoire ;
- l'opérateur `'+'` permet, selon le contexte, d'additionner des nombres, de concaténer des chaînes de caractères ou d'étendre des listes ;
- les listes possèdent une méthode `append`, mais pas les chaînes de caractères, qui ont une méthode `split` ;
- les chaînes de caractères et les listes sont « itérables » (on peut utiliser la syntaxe `'for el in var'` quand `var` est une chaîne de caractères ou une liste), mais pas les nombres.

Vocabulaire

La programmation orientée objet (POO) est un paradigme de programmation qui permet au développeur de créer de « nouveaux types », appelés **classes**. Dans ce contexte, une variable dont le « type » est associé à une classe est appelée **objet**. La création d'un objet s'appelle instanciation.
On peut résumer en disant que :

un objet est une instance d'une classe.

Une classe définit :

- la fonction appelée pour créer une instance : c'est le constructeur ;
- des variables propres à chaque instance : ce sont les attributs ;
- des fonctions qui s'appliquent à l'instance considérée : ce sont les méthodes.

L'ensemble des attributs et méthodes d'une classe en constituent les membres.

En python, les différents éléments de syntaxe sont les suivants :

- une classe est déclarée à l'aide du mot-clé `class`, dont le contenu doit être indenté ;
- le mot-clé `self` représente l'instance, il est utilisé pour déclarer et utiliser les membres de la classe ;
- on accède à un attribut d'un objet grâce à la syntaxe `objet.attribut` ;
- la définition d'une méthode prend toujours pour premier argument `self` ;
- on appelle une méthode grâce à la syntaxe `objet.methode(arguments)` ;
- le constructeur est la méthode `__init__(self)` ;
- l'instanciation d'un objet d'une classe `Truc` se fait à l'aide de la syntaxe `Truc(arguments)`.

Exercice 1

Achille et Hector décident de s'affronter en duel devant les murailles de Troie.
On dispose du programme suivant :

```
from random import randint

class Heros:

def __init__(self, nom, pv, res, dmax):
    print("Création du héros " + nom)
    self.nom = nom
    self.points_de_vie = pv
    self.resistance = res
    self.degats_max = dmax

def est_en_vie(self):
    return self.points_de_vie > 0

def attaque(self, autre):
    if self.est_en_vie():
        degats = randint(1, self.degats_max)
        autre.subit_attaque(self, degats)

def subit_attaque(self, autre, degats):
    if degats > self.resistance:
        self.points_de_vie -= degats
        print(self.nom + " perd "
              + str(degats) + " points de vie")
    else:
        print(autre.nom + " a raté son attaque contre " + self.nom)

    if not self.est_en_vie():
        print(autre.nom + " a tué " + self.nom)

achille = Heros("Achille", 100, 10, 30)
hector = Heros("Hector", 100, 15, 25)
paris = Heros("Pâris", 80, 8, 20)

# duel
while achille.est_en_vie() and hector.est_en_vie():
    achille.attaque(hector)
    hector.attaque(achille)
```

1. (a) La ligne `achille = Heros("Achille", 100, 10, 30)` permet de créer un héros : que représentent les quatre valeurs données entre parenthèses ?
   (b) Quelles sont les actions qu'un héros peut réaliser dans ce programme ?
   (c) Créer dans le programme un quatrième héros, `Ulysse`, possédant 90 points de vie, une résistance de 12 et des dégâts maximum de 25.
   (d) Le duel doit maintenant s'arrêter lorsqu'un des deux héros possède moins de 20 points de vie. Modifier le scénario en conséquence.
   (e) À la fin du combat, afficher le nom du héros qui possède encore le plus de points de vie et le nombre de tours nécessaires pour terminer le duel.

2. Les héros de la guerre de Troie peuvent disposer d'une capacité particulière. Par exemple :

   - Achille peut entrer dans une furie guerrière et infliger le maximum de dégâts ;
   - Ulysse peut récupérer des points de vie ;
   - Pâris peut effectuer une attaque précise qui ignore la résistance de son adversaire.

   (a) Ajouter à la classe `Heros` un attribut permettant de mémoriser la capacité spéciale du héros.
   (b) Modifier la création des héros afin de pouvoir préciser leur capacité spéciale.
   Par exemple : `achille = Heros("Achille", 100, 10, 30, "furie")`
   (c) Écrire une méthode `capacite_speciale` qui permet à un héros d'utiliser sa capacit éavec un comportement différent selon la capacité du héros :
      - `"furie"` : le héros inflige le maximum de dégâts à son adversaire ;
      - `"soin"` : le héros récupère 20 points de vie, sans dépasser 100 ;
      - `"precision"` : le héros inflige des dégâts sans tenir compte de la résistance de son adversaire.
   (d) Modifier le programme afin qu'à chaque tour, un héros ait une chance sur dix d'utiliser sa capacité spéciale plutôt que d'effectuer une attaque normale.
   (e) On souhaite maintenant ajouter une nouvelle capacité appelée `"bouclier"`.
   Cette capacité permet au héros de réduire de moitié les dégâts reçus lors de la prochaine attaque.
   Modifier la méthode `capacite_speciale` pour prendre en compte cette nouvelle capacité.

3. Les Grecs ne parviennent pas à prendre la cité. Ulysse propose alors une ruse : construire un immense cheval de bois et cacher des soldats à l'intérieur.

   (a) Écrire une classe `ChevalDeTroie` possédant
      - une capacité maximale ;
      - une liste de héros cachés à l'intérieur ;
      - une méthode permettant d'ajouter un héros dans le cheval.
   (b) Créer un cheval pouvant accueillir 5 héros et y cacher Achille, Ulysse et trois autres héros.
   (c) Écrire une méthode `est_complet` indiquant si le cheval contient déjà le nombre maximal de héros.

4. La bataille finale oppose maintenant deux armées : l'armée grecque et l'armée troyenne.

   Une armée est constituée de plusieurs héros. On souhaite créer une classe `Armee`.

   (a) Écrire le début de la classe `Armee` permettant de mémoriser
      - le nom de l'armée ;
      - la liste de ses héros.
   (b) Écrire une méthode `ajoute` permettant d'ajouter un héros à une armée.
   (c) Écrire une méthode `est_vaincue` qui renvoie `True` si tous les héros de l'armée sont morts et `False` sinon.
   (d) Créer les deux armées suivantes :
      - une armée grecque contenant Achille, Ulysse et plusieurs autres héros ;
      - une armée troyenne contenant Hector, Pâris et plusieurs autres héros.
   (e) Écrire un programme permettant de simuler la bataille entre les deux armées. À chaque tour, un héros vivant de l'armée grecque attaque un héros vivant de l'armée troyenne. Les Troyens ripostent ensuite.
   (f) Modifier le programme pour choisir aléatoirement le héros qui attaque.
   (g) Afficher à la fin de la bataille :
      - le nom de l'armée victorieuse ;
      - le nombre de héros encore en vie ;
      - le nombre de tours qu'a duré la bataille.

Définition

Une API (*Application Programming Interface* : interface de programmation d'application) consiste en la description des classes et des méthodes proposées par un module ou une bibliothèque.

Exercice 2

Compléter les méthodes de la classe `Angle` définie dans le fichier `angle.py`.

- `Angle(v)` (Constructeur)
  Paramètre : `v`, nombre représentant la mesure de l'angle en degrés (précondition : $0 \leqslant v \leqslant 180$).
  Crée un objet représentant un angle de mesure `v`. Déclenche une erreur si `v` n'est pas entre 0 et 180.
- `mesure_en_degres() -> float` : renvoie la mesure de l'angle en degrés.
- `est_aigu() -> bool` : Renvoie `True` si la mesure de l'angle est strictement inférieure à 90°, `False` sinon.
- `mesure_en_radians() -> float` : renvoie la mesure de l'angle en radians.
- `mesure_en_degres_minutes_secondes() -> tuple` renvoie un triplet `(degrés, minutes, secondes)` représentant la mesure de l'angle. Exemple : 12,5° correspond à `(12,30,0)`.
- `supplementaire() -> Angle` : renvoie un nouvel objet `Angle` représentant le supplémentaire de l'angle (le supplémentaire d'un angle de mesure v a pour mesure $180 - v$ degrés).
- `__str__() -> str` : renvoie une chaîne de caractères contenant la mesure de l'angle suivie du symbole °.
- `__add__(autre) -> Angle`
  Paramètre : `autre`, un objet de type `Angle`.
  Renvoie un nouvel objet `Angle` représentant la somme des deux angles (précondition : la somme doit être comprise entre 0° et 180°).
- `__sub__(autre) -> Angle`
  Paramètre : `autre`, un objet de type `Angle`.
  Renvoie un nouvel objet `Angle` représentant la différence entre les deux angles (précondition : la différence doit être comprise entre 0° et 180°).

Les méthodes statiques (hors programme)

Une méthode classique travaille sur un objet particulier. Elle reçoit automatiquement cet objet comme premier paramètre, c'est `self`. Par exemple, la méthode

```
def est_valide(self):
        ...
```

est appelée sur un objet : `instant.est_valide()`

Une méthode statique ne travaille sur aucun objet particulier. Elle ne reçoit donc pas de paramètre `self`. Elle est précédée du décorateur `staticmethod`.

```
@staticmethod
def est_bissextile(annee):
        ...
```

On peut alors l'appeler directement à partir de la classe : `Instant.est_bissextile(2024)`

Exercice 3

Compléter les méthodes de la classe `Instant` définie dans le fichier `instant.py` qui permet de représenter et manipuler des instants au format `jj/mm/aaaa:hh:mm:ss`.